

LOW-LEVEL DESIGN DOCUMENT

Version 1.0



Chef Bono

Course	CMPE 492 / CS 492
Team	Chef Bono Team
Members	Berkan Gökgöz Buse Şahin Yelda Sıla Mumcu Eren Serdar
Date	20/03/2026
Version	1.0

1. Introduction

1.1 Object Design Trade-offs

Here are the key trade-offs when creating the modules in our project.

Trade-off	Option A	Option B	Decision & Rationale
Reliability vs Speed	Create an ai agent that can imitate multiple roles based on user input	Use function calling to decide which type of ai agent must process the user input	Initially implemented option A to reduce delay to a single request but single agent with multiple assignments did not perform well, so we switched to option B and degraded the delay time to 2 requests time at least.
Storage vs Performance	To perform vector searches of the recipe database based on user input, save 3072-dimensional (original gemini embed size) vectors for each recipe.	To perform vector searches of the recipe database based on user input, save 256-dimensional vectors for each recipe.	Even though option A performs way better, the storage would increase too much for the, give or take, 22k recipes we have. We used PCA library to decrease the dimension to 256. We saved space but it is less reliably compared to the original embed size 3072 because the embeddings lost some meaning.
Cost vs Reliability	Using the models by running them locally.	Using the models through API calls.	We decided to go with API calls. Even though they require internet connection and not as reliable (server congestion, unexpected changes in the api, etc.) they can work on any computer. Which is less costly and what we aimed for.

1.2 Interface Documentation Guidelines

All class interfaces in this document follow the conventions listed below:

- Method signatures are written as: returnType methodName(paramType paramName, ...)
- Exceptions are listed explicitly for each method that may throw them
- Preconditions and postconditions are stated where behaviour depends on system state
- Visibility modifiers: + (public), - (private), # (protected)

1.3 Engineering Standards

The following engineering standards were applied throughout this design phase:

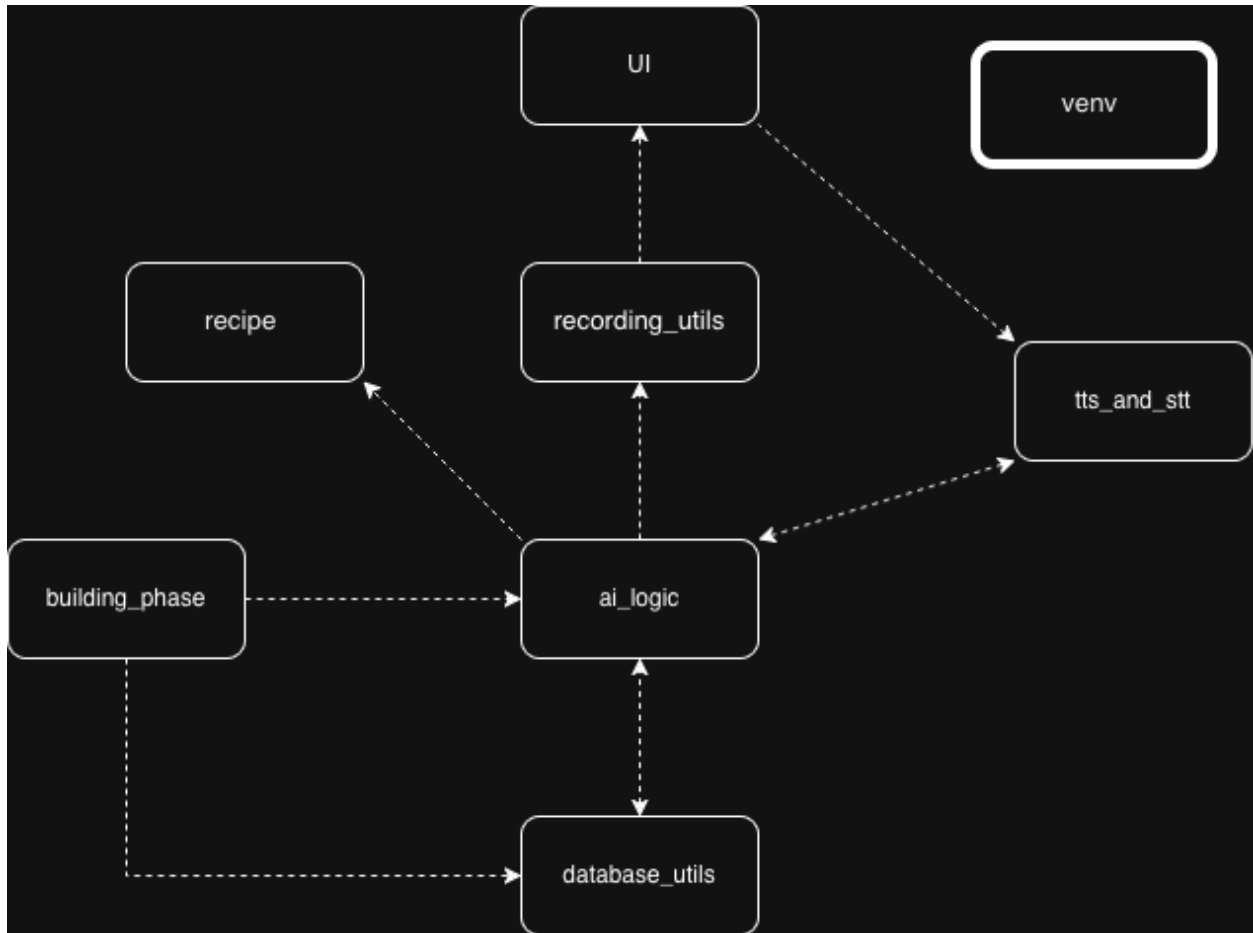
Standard	Description	Application in This Project
UML 2.x	Unified Modeling Language for class, sequence, and package diagrams	<i>[Used for all structural diagrams in Section 3]</i>
IEEE 1016-2009	IEEE Standard for Information Technology — Software Design Descriptions	<i>[Report structure follows this standard]</i>
ACM Code of Ethics	Professional and ethical obligations of software engineers	<i>[Considered in data privacy and security design decisions]</i>

1.4 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
HLD	High-Level Design — the architectural design document produced in CMPE 491
LLD	Low-Level Design — the detailed object design document (this document)
UML	Unified Modeling Language
OOP	Object-Oriented Programming
<i>AI Agent / Multi-Agent</i>	<i>An autonomous software component that perceives input and selects actions to achieve a goal. Chef Bono uses a multi-agent architecture (Section 1.1) where a function-calling router delegates to specialised agents for SQL retrieval, recipe selection, step-by-step guidance, and general conversation.</i>

2. Packages

This section describes the logical grouping of classes into packages. The package diagram below illustrates dependencies between packages and their alignment to the subsystems identified in the HLD.



Package Name	Responsibility	Key Classes/Files contained
UI	<i>Handles all user interface rendering.</i>	<i>Interface.py</i>
building_phase	<i>Responsible of scraping recipes, classifying them, then loading them to the database</i>	<i>scrap_recipes.py, row_adder.py</i>
tts_and_stt	<i>Getting sound input from the user and delivering sound output to the user</i>	<i>stt.py, tts.py</i>
database_utils	<i>CRUD operations, creating database</i>	<i>clear_database.py database_structure.sql meal_class_images recipes.db create_database.py db_query_handler.py read_database.py scraped_recipes.jsonl</i>

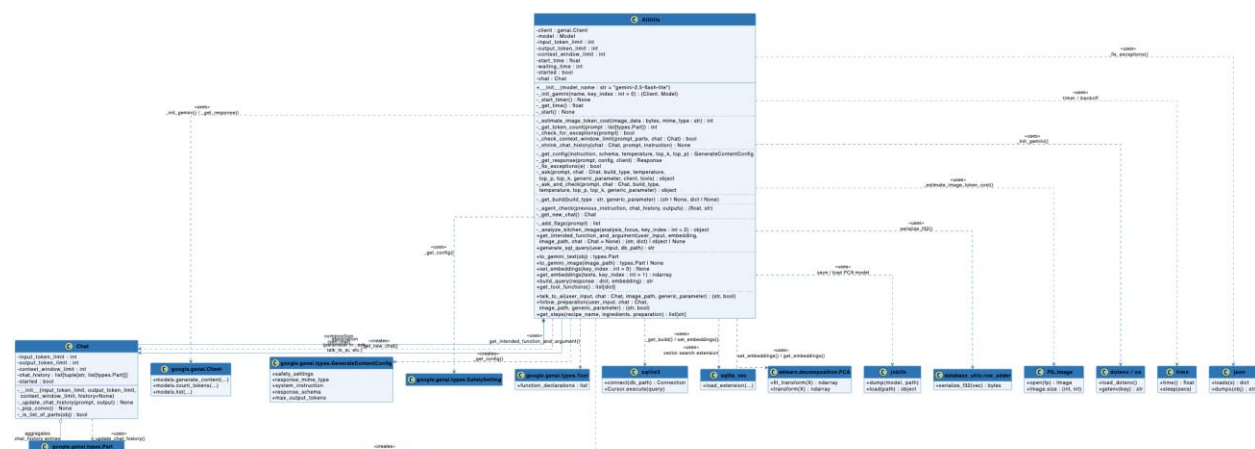
recipe	<i>Recipe object</i>	<i>recipe_manager.py</i>
ai_logic	<i>LLM logic, image analysis, function calling, text embedding</i>	<i>ai_utils.py pca_model.joblib</i>
recording_utils	<i>Capture the momentaneous frame</i>	<i>camera.py current_frame.jpg</i>
venv	<i>Python's virtual environment that holds external packages</i>	<i>Lib directory</i>

Venv is a dependency for every packages

3. Class Interfaces

3.1 Subsystem AI logic

This section provides a detailed interface specification for all classes in the AI utility subsystem. Two classes are documented: Chat, which manages multi-turn conversational state, and AIUtils, which wraps the Google Gemini API to provide prompting, tool-calling, token management, embedding, and SQL-generation capabilities.



3.1.1 Component: Chat

Package: ai_utils | Subsystem: Conversation State Management

Chat is a lightweight container that holds the multi-turn conversation history used by AIUtils to maintain context across Gemini API calls. It normalises prompts and responses into lists of `google.genai.types.Part` and provides utilities to append or remove turns.

Design Pattern

Data Object / Value Holder — Chat is a passive data container with no business logic; it delegates all Gemini API concerns to AIUtils. This separation of state from behavior follows the Single Responsibility Principle.

Attributes

Vis.	Name	Type	Description
+	<code>input_token_limit</code>	<code>int</code>	Maximum input tokens allowed by the model.
+	<code>output_token_limit</code>	<code>int</code>	Maximum output tokens allowed by the model.
+	<code>context_window_limit</code>	<code>int</code>	Sum of <code>input_token_limit</code> + <code>output_token_limit</code> .
-	<code>chat_history</code>	<code>list[tuple[str, list[Part]]]</code>	Conversation stored as alternating ("user", [Part...]) and ("assistant", [Part...]) pairs.
+	<code>started</code>	<code>bool</code>	Flag indicating chat timing/state. Not actively used within this class.

Methods

Vis.	Signature	Returns	Description
+	<code>__init__(input_token_limit, output_token_limit, context_window_limit, history=None)</code>	None	Initialises token limits and optionally seeds <code>chat_history</code> from a provided history list.
-	<code>_update_chat_history(prompt, output)</code>	None	Normalises prompt and output into <code>list[Part]</code> if needed, then appends ("user", parts) and ("assistant", parts) to <code>chat_history</code> .
-	<code>_pop_convo()</code>	None	Removes the last user+assistant turn (2 entries) from <code>chat_history</code> . Prints a message and returns early if fewer than 2 entries exist.
-	<code>_is_list_of_parts(obj) -> bool</code>	bool	Returns True if <code>obj</code> is a list and every element is

			an instance of google.genai.types.Part.
--	--	--	--

Class Relationships

Relationship	With	Description
Aggregation (stores)	google.genai.types.Part	chat_history stores prompts and outputs as lists of types.Part, structured as ("user", [Part...]) and ("assistant", [Part...]) pairs.
Dependency (uses)	google.genai.types.Part	_update_chat_history() wraps raw strings into Part.from_text() when the input is not already a list of Parts.

Sequence Diagram

[Insert Sequence Diagram: AIUtils._ask() → Chat._update_chat_history()]

3.1.2 Component: AIUtils

Package: ai_utils | Subsystem: AI / LLM Integration

AIUtils is the central facade for all Google Gemini interactions. It wraps prompting, schema-constrained JSON outputs, tool/function calling, token counting, context-window management, embedding generation with PCA compression, SQL query generation, and a set of application-level conversational features (chef talk, recipe building, kitchen image analysis).

Design Patterns

- Facade — AIUtils presents a single simplified interface over the google.genai SDK, sqlite, sklearn PCA, and joblib.
- Strategy — build_type parameter in _get_build() selects different instruction+schema strategies at runtime without branching in the caller.
- Template Method — _ask_and_check() provides a fixed retry/evaluation skeleton while delegating scoring to _agent_check().
- Retry / Circuit-breaker — _fix_exceptions() implements rate-limit backoff (429) and service-unavailable recovery (503).

Core Attributes

Vis.	Name	Type	Description
-	client	genai.Client	Initialised via _init_gemini(). Used for all generate_content and count_tokens calls.

-	model	Model	Matched from client.models.list(). Provides name, input_token_limit, output_token_limit.
+	input_token_limit	int	Derived from model.input_token_limit.
+	output_token_limit	int	Derived from model.output_token_limit.
+	context_window_limit	int	input_token_limit + output_token_limit.
-	start_time	float	Epoch time when timer was last started. 0 means not started.
-	waiting_time	int	Accumulated wait time used in _fix_exceptions() backoff calculations.
-	started	bool	Set True by _start(). Not essential outside timer helpers.
-	chat	Chat	Default Chat instance created in __init__ with model limits. Used as the default conversation context.

A) Initialisation / Timing

Vis.	Signature	Returns	Description
-	<code>__init__(model_name="gemini-2.5-flash-lite")</code>	None	Initialises Gemini client and model, token limits, timer state, and a default Chat instance.
-	<code>_init_gemini(name, key_index=0) -> (client, model)</code>	tuple	Loads env vars, reads GOOGLE_API_KEY{key_index}, creates genai.Client, lists models (page_size=5), returns exact match for "models/{name}".
-	<code>_start_timer()</code>	None	Sets self.start_time to the current epoch time.
-	<code>_get_time() -> float</code>	float	Returns elapsed seconds since self.start_time.
-	<code>_start()</code>	None	Sets started=True and calls _start_timer().

B) Token Counting & Context Management

Vis.	Signature	Returns	Description
-	<code>_estimate_image_token_cost(image_data: bytes, mime_type: str) -> int</code>	int	Estimates token cost for an image using rough tiling logic via PIL. Falls back to 258 tokens on error.

-	<code>_get_token_count(prompt: list[Part]) -> int</code>	int	Sums token count for text parts via <code>client.models.count_tokens()</code> plus estimated image cost for image parts.
-	<code>_check_for_exceptions(prompt) -> bool</code>	bool	Returns True (and prints a message) if prompt token count exceeds <code>input_token_limit</code> .
-	<code>_check_context_window_limit(prompt_parts, chat) -> bool</code>	bool	Returns True if <code>tokens(prompt_parts) + chat.output_token_limit</code> exceeds <code>chat.context_window_limit</code> .
-	<code>_shrink_chat_history(chat, prompt, instruction)</code>	None	Pops turns from chat history until token usage falls below ~80% of <code>input_token_limit</code> .

C) Gemini Request Building & Execution

Vis	Signature	Returns	Description
.			
-	<code>_get_config(instruction=None, schema=None, temperature=0.7, top_k=40, top_p=0.9)</code>	GenerateContentConfig	Builds a <code>GenerateContentConfig</code> : <code>BLOCK_NONE</code> safety, JSON mime type, system instruction as Part list, response_schema, max_output_tokens.
-	<code>_get_response(prompt, config, client)</code>	Response	Calls <code>client.models.generate_content()</code> . Starts timer if needed. On exception calls <code>_fix_exceptions</code> and retries.
-	<code>_fix_exceptions(e) -> bool</code>	bool	Handles API errors by code: 429 parses <code>retryDelay</code> and sleeps; 503 sleeps up to 60 s. Returns True to signal retry, False otherwise.
-	<code>_ask(prompt, chat, build_type=None, temperature=0.7, top_p=0.9, top_k=40, generic_parameter=None, client=None, tools=None)</code>	parsed obj tuple	Main send-prompt entry. Fetches instruction+schema, builds config, flattens history+prompt, checks context, calls <code>_get_response</code> . Returns function call tuple or parsed object.
-	<code>_ask_and_check(prompt, chat, build_type=None, temperature=0.7, top_p=0.9,</code>	best response	Calls <code>_ask</code> up to 3 times and evaluates each result via <code>_agent_check</code> . Keeps the

	<code>top_k=40, generic_parameter=None)</code>		response with score ≥ 0.7 or the highest scoring attempt.
--	--	--	--

D) Build Types — Instruction & Schema Selection

`_get_build(build_type, generic_parameter=None)` returns an (instruction: str|None, schema: dict|None) pair used by `_get_config()`. The table below documents each supported `build_type` value.

build_type	Intent	Output Schema Fields
"agent_check"	Evaluator scoring a response attempt.	{score, feed_back}
"kitchen_image_analysis"	Structured analysis of a kitchen image.	{food_items, supplies, tips}
"add_flags"	Recipe classification, nutrition scores, meal class enum.	{flags, nutrition_scores, meal_class}
"query"	Structured SQL keyword-block from sqlite schema + vector-search rules.	{SELECT, FROM, WHERE, MATCH, LIMIT, ...}
"decide_intent"	Chooses index [0..3] and parameters for function/tool routing.	{index, parameters}
"talk"	Casual chef conversational response.	{chef's_response, keep_convo_alive}
"follow_preparation"	Step-by-step cooking guidance.	{cooking_instruction, keep_convo_alive}
"build_recipe"	Full recipe step generation.	{cooking&preparation_steps: [...]}

E) Agent Evaluation

Vis.	Signature	Returns	Description
-	<code>_agent_check(previous_instruction, chat_history, outputs) -> (score, feed_back)</code>	tuple[float, str]	Builds a prompt with system instruction, chat history, and outputs. Calls <code>_ask(build_type="agent_check")</code> on a fresh Chat. Returns (score, feed_back).
-	<code>_get_new_chat() -> Chat</code>	Chat	Constructs and returns a fresh Chat instance using current token limits.

F) Application Features

Vis	Signature	Returns	Description
-	<code>_add_flags(prompt)</code>	list	Sends recipes to <code>build_type="add_flags"</code> and inserts an embedding field of 256 zeros into each result.
-	<code>_analyze_kitchen_image(analysis_focus, key_index=2)</code>	parsed obj	Prompts with fixed file "current_frame.jpg" and <code>build_type="kitchen_image_analysis"</code> using client initialised at <code>key_index</code> .
-	<code>get_intended_function_and_argument(user_input, embedding, image_path, chat=None)</code>	tuple parsed None	Builds prompt with <code>user_input</code> and kitchen image, enables tool calling via <code>types.Tool(function_declarations=...)</code> . Returns (function_name, args) tuple or parsed JSON.
-	<code>generate_sql_query(user_input, db_path)</code>	str	Gets embedding for <code>user_input</code> , calls <code>_ask(build_type="query")</code> with DB schema context, returns SQL string from <code>build_query()</code> .

G) Converters / Embeddings / SQL Building

Vis.	Signature	Returns	Description
+	<code>to_gemini_text(obj) -> Part</code>	Part	Converts dict/list → JSON string Part, str → text Part, None → empty string Part.
+	<code>to_gemini_image(image_path) -> Part None</code>	Part None	Reads image bytes and returns <code>Part.from_bytes</code> with inferred MIME type. Returns None on file read error.
-	<code>set_embeddings(key_index=0)</code>	None	Loads recipes from sqlite, creates embeddings via gemini-embedding-001, PCA-reduces to ≤ 256 dims, saves PCA model, writes into <code>recipes_vec</code> .
-	<code>get_embeddings(texts, key_index=1) -> vector</code>	list	Embeds texts via gemini-embedding-001, loads PCA model, transforms. Returns only the first transformed vector.
-	<code>build_query(response: dict, embedding) -> str</code>	str	Converts structured keyword-block JSON into

			a SQL string. Special-cases JOIN for FROM="recipes".
+	<code>get_tool_functions() -> list[dict]</code>	list	Returns 4 function declarations: generate_sql, select_recipe, follow_preparation, respond_to_user.

H) Public Conversational Wrappers

Vis.	Signature	Returns	Description
+	<code>talk_to_ai(user_input, chat, image_path=None, generic_parameter=None) -> tuple</code>	tuple	Calls <code>_ask(build_type="talk")</code> and returns (chef_response, keep_convo_alive) from parsed result.
+	<code>follow_preparation(user_input, chat, image_path=None, generic_parameter=None) -> tuple</code>	tuple	Calls <code>_ask(build_type="follow_preparation")</code> and returns (chef_response, keep_convo_alive).
+	<code>get_steps(recipe_name, ingredients, preparation) -> list[str]</code>	list[str]	Calls <code>_ask(build_type="build_recipe")</code> and returns the "cooking&preparation_steps" list.

Class Relationships

Relationship	With	Description
Composition (has-a)	Chat	AIUtils creates and owns a Chat instance as self.chat to store conversation state.
Factory (creates)	Chat	<code>_get_new_chat()</code> constructs and returns a fresh Chat object for isolated conversations.
Association (parameter)	Chat	<code>_ask()</code> , <code>talk_to_ai()</code> , <code>follow_preparation()</code> , and <code>get_intended_function_and_argument()</code> accept a Chat instance to read and update chat_history.
Dependency (returns)	google.genai.types.Part	<code>to_gemini_text()</code> and <code>to_gemini_image()</code> produce Part objects used as Gemini inputs.
Dependency (uses)	google.genai.Client	<code>_init_gemini()</code> constructs genai.Client; <code>_get_response()</code> calls <code>client.models.generate_content()</code> ; token

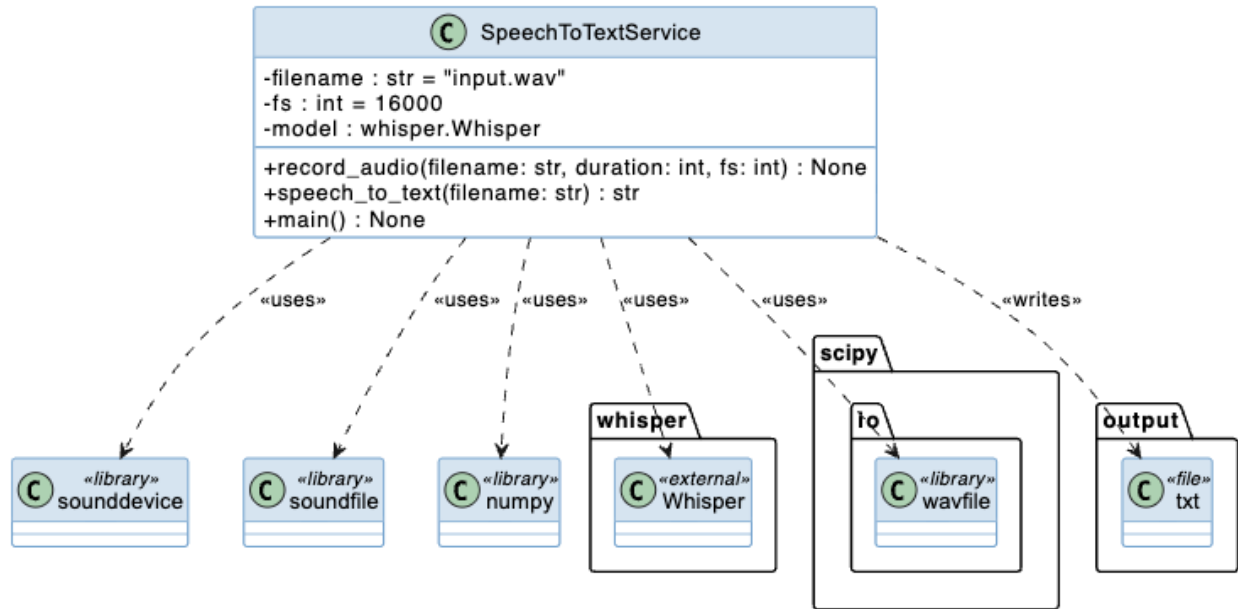
		counting uses client.models.count_tokens().
Dependency (uses)	google.genai.types.GenerateContentConfig	_get_config() builds a GenerateContentConfig used in every model call.
Dependency (uses)	google.genai.types.SafetySetting	_get_config() configures multiple SafetySetting entries (all BLOCK_NONE).
Dependency (uses)	google.genai.types.Tool	get_intended_function_and_argument() creates a types.Tool for function-calling behaviour.
Dependency (uses)	sqlite3.Connection / Cursor	_get_build("query"/"decide_intent") introspects DB schema; set_embeddings() reads and writes recipes / recipes_vec.
Dependency (uses)	sqlite_vec	_get_build() and set_embeddings() load the sqlite_vec extension for vector search and storage.
Dependency (uses)	sklearn.decomposition.PCA	set_embeddings() fits PCA and transforms embeddings; get_embeddings() uses the saved PCA to transform new embeddings.
Dependency (uses)	joblib	set_embeddings() saves PCA model to pca_model.joblib; get_embeddings() loads it.
Dependency (uses)	database_utils.row_adder	set_embeddings() calls row_adder.serialize_f32() to serialise embeddings into the sqlite vector table.

3.2 Speech-to-Text and Text-to-Speech Subsystem

This component belongs to the Speech Processing subsystem. It provides two-way audio communication: converting speech to text (STT) and synthesizing text to speech (TTS).

[Briefly describe this component's purpose and its role in the overall architecture]

UML Class Diagram



Attributes:

Visibility	Name	Type	Description
-	[attributeName]	[String]	[Describe the attribute's purpose]
-	[attributeName]	[int]	[Describe the attribute's purpose]

Methods:

Visibility	Signature	Return Type	Description	Exceptions
+	[methodName(param: Type)]	[ReturnType]	[What the method does]	[ExceptionType if...]
+	[methodName()]	[void]	[What the method does]	[None]

Class Relationships

Relationship Type	With Class	Description
[e.g., Association]	[OtherClassName]	[Describe the nature of the relationship]
[e.g., Inheritance]	[ParentClassName]	[Extends / implements for purpose...]

3.2.1 Speech-to-Text (STT) Module — stt.py

Converts microphone input into a text string using the OpenAI Whisper model. Audio is captured via sounddevice, saved as a WAV file, and transcribed locally using a Whisper base model checkpoint. The transcription language is fixed to Turkish (tr).

UML Class Diagram — SpeechToTextService

Class: SpeechToTextService

Design pattern applied: Facade — hides the multi-step pipeline (record, read, normalise, transcribe, write) behind two simple public methods.

Attributes:

Visibility	Name	Type	Description
-	filename	str	Path to the WAV file used for recording and transcription (default: input.wav)
-	duration	int	Recording duration in seconds (default: 5)
-	fs	int	Sampling frequency in Hz (default: 16000)
-	model	whisper.Whisper	Whisper model instance; loaded lazily at transcription time

Methods:

Visibility	Signature	Return	Description	Exceptions
+	record_audio(filename, duration, fs)	None	Records audio from the default mic for the given duration and saves it as a WAV file at filename.	IOError if file cannot be written
+	speech_to_text(filename) -> str	str	Loads Whisper base, reads WAV as float32, collapses multi-channel to mono, runs transcription with language=tr, writes result to output.txt, and returns the text.	FileNotFoundError; RuntimeError on model failure
+	main()	None	Entry point: calls record_audio() then speech_to_text(). Sets fallback string if transcription is empty.	None

Key Implementation Notes:

- Multi-channel audio collapsed to mono via: `audio = audio.mean(axis=1)`
- Audio explicitly cast to float32 to satisfy Whisper's input contract
- Transcription persisted to `output.txt` for consumption by downstream subsystems
- Fallback text "Seni anlayamadım" is returned when transcription yields an empty string

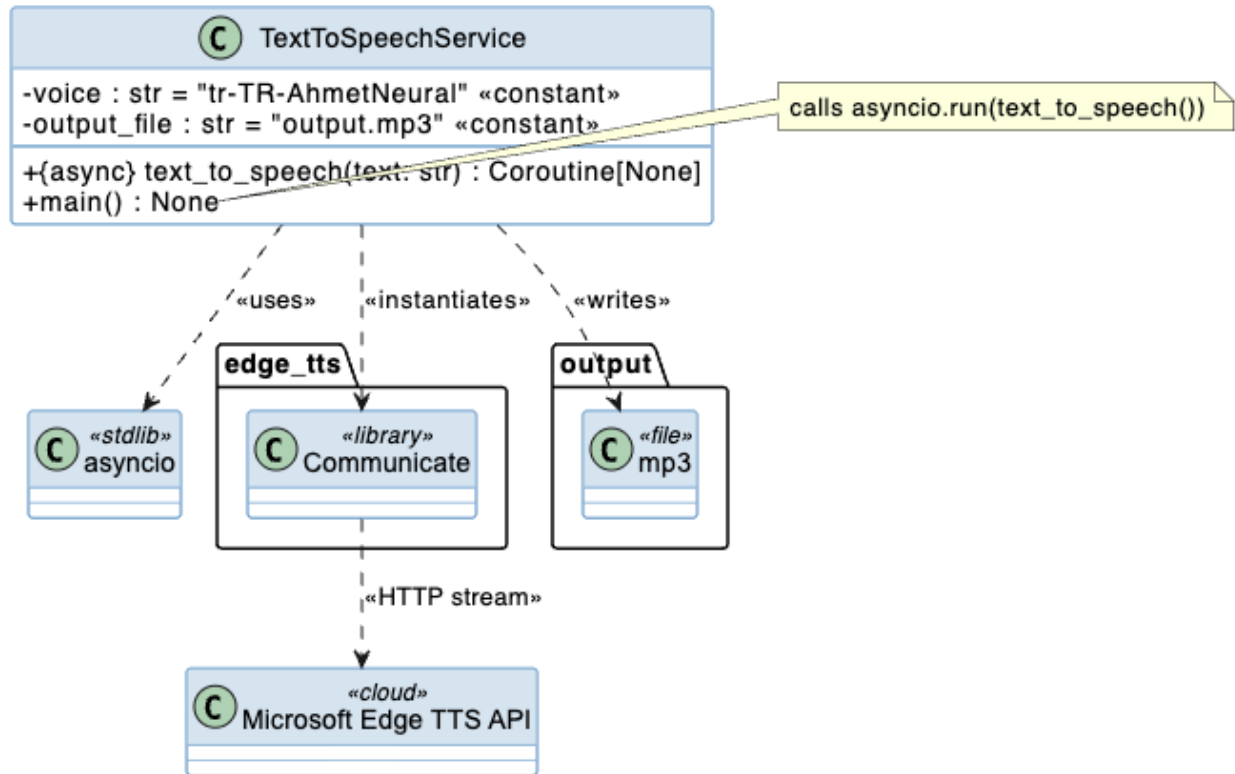
External Dependencies:

Library	Source	Role
sounddevice	PyPI	Microphone audio capture via PortAudio bindings
scipy.io.wavfile	SciPy / PyPI	Writing raw NumPy arrays as WAV files
openai-whisper	OpenAI / PyPI	Local speech recognition model (base checkpoint)
soundfile	PyPI	Reading WAV files with float32 dtype support
numpy	PyPI	Array manipulation (mono conversion, dtype casting)

3.2.2 Text-to-Speech (TTS) Module — tts.py

Converts a text string into an MP3 audio file using Microsoft Edge TTS via the `edge-tts` Python library. The voice is fixed to `tr-TR-AhmetNeural` (Turkish male neural voice). The module is fully asynchronous, driven by Python's `asyncio` event loop.

UML Class Diagram — TextToSpeechService



Class: TextToSpeechService

Design pattern applied: Async Wrapper / Facade — exposes a synchronous entry point (`main`) over an asynchronous coroutine using `asyncio.run()`, shielding callers from event loop management.

Attributes:

Visibility	Name	Type	Description
-	voice	str (constant)	Edge TTS voice ID: tr-TR-AhmetNeural (Turkish male neural voice)
-	output_file	str (constant)	Output file path: output.mp3 (written to current working directory)

Methods:

Visibility	Signature	Return	Description	Exceptions
+	<code>text_to_speech(text: str)</code>	<code>Coroutine[None]</code>	Async method. Creates <code>edge_tts.Communicate</code> with the given text and tr-TR-AhmetNeural voice, then awaits <code>communicate.save()</code> to write <code>output.mp3</code> .	<code>NoAudioReceived</code> if TTS fails; <code>IOError</code> on file write failure

+	main()	None	Synchronous entry point. Defines input text and calls <code>asyncio.run(text_to_speech(text))</code> to execute the async coroutine.	None
---	--------	------	--	------

Key Implementation Notes:

- ⊄ Requires an active internet connection — edge-tts streams synthesis from Microsoft's cloud service
- ⊄ Output is always written to output.mp3 in the working directory; callers must read this file after invocation
- ⊄ Voice is hardcoded to tr-TR-AhmetNeural; parameterisation is required to support other languages
- ⊄ `asyncio.run()` creates a new event loop — calling from within an existing async context will raise `RuntimeError`

External Dependencies:

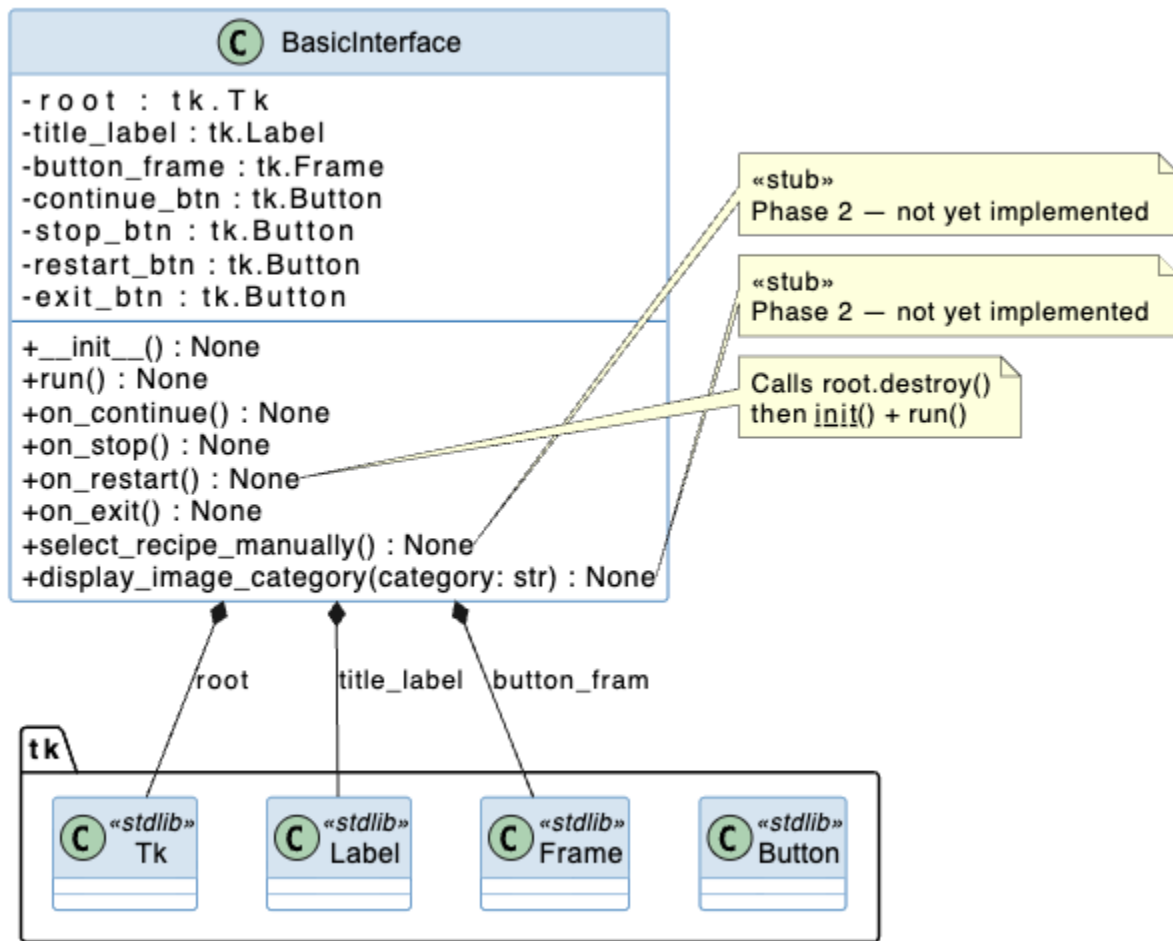
Library	Source	Role
edge-tts	PyPI	Python client for Microsoft Edge cloud-based neural TTS service
asyncio	Python stdlib	Event loop management for async coroutine execution

3.3 User Interface Subsystem — BasicInterface

This component belongs to the User Interface subsystem. BasicInterface is a tkinter-based desktop GUI that serves as the primary control panel for the Chef Bono application. It was identified during object design as the presentation layer, responsible for rendering the application window and routing user button events to the underlying AI and session logic.

UML Class Diagram

anager

**Class: BasicInterface**

Design pattern applied: Facade — wraps the entire tkinter widget hierarchy and event loop behind a single public `run()` method, presenting a minimal interface to callers. Button callbacks act as event handlers following the Observer-like Command pattern: each button is bound to a dedicated handler method, keeping event logic decoupled from widget construction.

Attributes:

Visibility	Name	Type	Description
-	root	tk.Tk	The main application window instance; serves as the root widget for all child elements
-	title_label	tk.Label	Displays the application name “Chef Bono” at the top of the window
-	button_frame	tk.Frame	Container frame that groups and centres all action buttons
-	continue_btn	tk.Button	Triggers <code>on_continue()</code> ; signals the AI loop to proceed with the next step

-	stop_btn	tk.Button	Triggers on_stop(); pauses or halts the current AI session
-	restart_btn	tk.Button	Triggers on_restart(); destroys and re-initialises the window and session
-	exit_btn	tk.Button	Triggers on_exit(); calls root.quit() to terminate the application

Methods:

Visibility	Signature	Return	Description	Exceptions
+	__init__(self)	None	Constructs the root window, configures geometry (400x400) and background (black), instantiates and packs all labels, frames, and buttons with their command bindings.	tk.TclError if display unavailable
+	run(self)	None	Enters the tkinter main event loop via root.mainloop(). Blocks until the window is closed.	None
+	on_continue(self)	None	Event handler for the Continue button. Currently prints a debug message; intended to signal the AI logic module to resume processing.	None
+	on_stop(self)	None	Event handler for the Stop button. Currently prints a debug message; intended to pause the active AI session.	None
+	on_restart(self)	None	Event handler for the Restart button. Destroys the current window, re-calls __init__() to rebuild the UI, then calls run() to re-enter the event loop.	None
+	on_exit(self)	None	Event handler for the Exit button. Calls root.quit() to terminate	None

			the mainloop and close the application.	
+	select_recipe_manually(self)	None	Stub method reserved for future implementation. Will present a manual recipe selection dialogue to the user.	NotImplementedError (stub)
+	display_image_category(self, category: str)	None	Stub method reserved for future implementation. Will display visual content for the given recipe category.	NotImplementedError (stub)

Key Implementation Notes:

- ⊄ The window is fixed at 400x400 pixels with a black background (bg="black") and white title text, establishing the Chef Bono visual identity
- ⊄ on_restart() uses a self-reinitialisation pattern: it calls self.root.destroy() followed by self.__init__() and self.run(), effectively recreating the entire widget tree in place
- ⊄ select_recipe_manually() and display_image_category() are declared as stubs with bare break statements; these are placeholders for Phase 2 implementation and will raise errors if called in the current state
- ⊄ AIUtils is imported from ai_logic but not yet instantiated within BasicInterface — the dependency is declared at module level, indicating planned integration in a future iteration
- ⊄ All buttons share a consistent style: width=15, Arial 14pt font, bg="gray20", packed with pady=10 inside button_frame for uniform spacing

External Dependencies:

Library / Module	Source	Role
tkinter (tk)	Python stdlib	GUI framework: provides Tk, Label, Frame, Button, and mainloop event handling
ai_logic.AIUtils	Internal module	AI processing utility class; imported for planned integration with button event handlers

Widget Hierarchy

The following tree shows the parent-child containment of all tkinter widgets constructed in __init__():

Level	Widget	Type	Key Properties
-------	--------	------	----------------

0	root	tk.Tk	title=Chef Bono, geometry=400x400, bg=black
1	title_label	tk.Label	text=Chef Bono, font=Arial 24 bold, fg=white, bg=black, pady=30
1	button_frame	tk.Frame	bg=black, expand=True (centres vertically)
2	continue_btn	tk.Button	command=on_continue, width=15, pady=10
2	stop_btn	tk.Button	command=on_stop, width=15, pady=10
2	restart_btn	tk.Button	command=on_restart, width=15, pady=10
2	exit_btn	tk.Button	command=on_exit, width=15, pady=10

Class Relationships

Relationship	From	To	Description
Composition	BasicInterface	tk.Tk (root)	BasicInterface owns and manages the root window lifecycle
Composition	BasicInterface	tk.Label, tk.Frame, tk.Button	All widgets are owned by BasicInterface and destroyed with it
Dependency (import)	BasicInterface	ai_logic.AIUtils	AIUtils imported at module level; direct usage planned for Phase 2
Event binding	tk.Button (continue_btn)	BasicInterface.on_continue()	Button command binding routes click events to handler
Event binding	tk.Button (stop_btn)	BasicInterface.on_stop()	Button command binding routes click events to handler
Event binding	tk.Button (restart_btn)	BasicInterface.on_restart()	Button command binding triggers UI teardown and reinitialisation
Event binding	tk.Button (exit_btn)	BasicInterface.on_exit()	Button command binding terminates the mainloop

Design Patterns Summary:

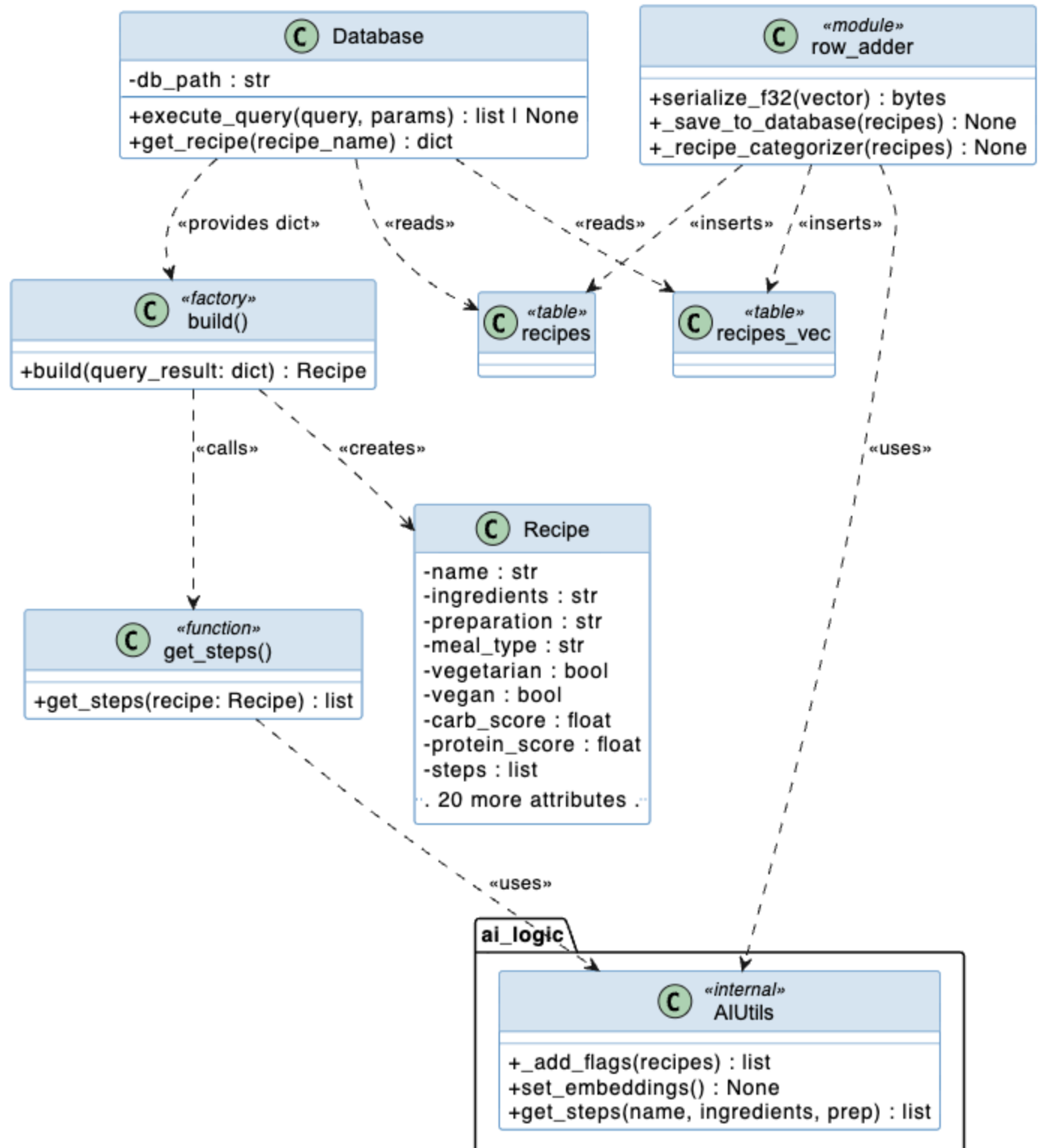
Pattern	Where Applied	Justification
Facade	BasicInterface.run()	Hides the full tkinter widget construction and event loop behind a single run() entry point, simplifying usage by the __main__ caller
Command (partial)	Button → on_X() bindings	Each button is bound to a dedicated handler method, decoupling the widget from the action

		logic and making it easy to extend handlers independently
Self-reinitialisation	on_restart()	Destroys and rebuilds the UI in place without requiring an external orchestrator to manage the restart lifecycle

3.4 Database and Recipe Management Subsystem

This subsystem belongs to the Data Layer and is composed of three collaborating modules: `db_query_handler.py` (raw SQL access), `recipe_manager.py` (domain object construction), and `row_adder.py` (batch ingestion pipeline). Together they implement the full CRUD lifecycle for recipe data stored in a SQLite database augmented with vector similarity search via `sqlite_vec`. The subsystem bridges the AI-generated recipe metadata from the ingestion pipeline to the structured domain objects consumed by the rest of the application.

UML Class Diagram



3.4.1 Class: Database — db_query_handler.py

Provides a thin, reusable wrapper around SQLite connections. Every public method opens a fresh connection, executes its query, commits, and closes — avoiding long-lived connection state. The `sqlite_vec` extension is loaded on each connection to enable vector operations.

Design pattern applied: Repository — encapsulates all raw data-access logic behind two public methods, decoupling callers from SQL syntax and connection management.

Attributes:

Visibility	Name	Type	Description
-	<code>db_path</code>	<code>str</code>	Filesystem path to the SQLite database file (e.g. <code>recipes.db</code>); stored at construction time and reused for every connection

Methods:

Visibility	Signature	Return	Description	Exceptions
+	<code>__init__(self, db_path: str)</code>	None	Stores the database path. Does not open a connection at construction time.	None
+	<code>execute_query(self, query: str, params=None)</code>	list None	Opens a connection, loads <code>sqlite_vec</code> , executes the parameterised query, fetches all results, commits, and closes. Returns the result rows or None on error.	<code>sqlite3.Error</code> caught internally; prints error and returns None
+	<code>get_recipe(self, recipe_name: str)</code>	dict	Fetches a single recipe row by name. Deserialises binary fields (e.g. embeddings) via <code>read_database.serialize_bytes()</code> and returns a column-keyed dict.	<code>sqlite3.Error</code> (uncaught); returns empty dict if name not found

Key Implementation Notes:

- ⚡ `sqlite_vec` is loaded via `conn.enable_load_extension(True) / sqlite_vec.load(conn) / conn.enable_load_extension(False)` on every connection, enabling vector similarity queries on the `recipes_vec` virtual table
- ⚡ `execute_query()` uses parameterised queries (params tuple) to prevent SQL injection; `get_recipe()` currently uses f-string interpolation — a known security gap flagged for remediation
- ⚡ Connection lifecycle is fully contained within each method call (open → use → close), preventing connection leaks in long-running sessions
- ⚡ `read_database.serialize_bytes()` is called per field in `get_recipe()` to handle the binary-serialised embedding vector stored in the `recipes` table

3.4.2 Class: Recipe — recipe_manager.py

A plain domain object (data class) representing a single recipe with all its nutritional, dietary, and metadata attributes. It is constructed by the module-level build() factory function from a database query result dict and populated with AI-generated step-by-step cooking instructions via get_steps().

Design pattern applied: Factory Method (build()) — the Recipe class itself has no knowledge of the database or AI layer; all construction logic lives in the build() function, which maps raw query results to typed attributes and delegates step generation to the AI model.

Attributes:

Visibility	Name	Type	Default	Description
-	id	int None	None	Primary key from the recipes table
-	name	str None	None	Recipe display name
-	ingredients	str None	None	Raw ingredients string as stored in the database
-	preparation	str None	None	Raw preparation instructions string
-	meal_type	str None	None	Category: e.g. breakfast, lunch, dinner, snack
-	main_ingredient	str None	None	Primary ingredient identifier
-	nationality	str None	None	Cuisine origin / nationality tag
-	vegetarian	bool	False	Dietary flag: contains no meat
-	vegan	bool	False	Dietary flag: contains no animal products
-	gluten_free	bool	False	Dietary flag: no gluten-containing ingredients
-	dairy_free	bool	False	Dietary flag: no dairy products
-	nut_free	bool	False	Dietary flag: no nut ingredients
-	high_fiber	bool	False	Nutritional flag: high dietary fibre content
-	weight_loss_diet	bool	False	Goal flag: suitable for weight-loss programmes

-	muscle_gain_bulking	bool	False	Goal flag: suitable for muscle gain / bulking
-	balanced_healthy	bool	False	Goal flag: balanced macronutrient profile
-	carb_score	float	0.0	Normalised carbohydrate content score (0–1)
-	protein_score	float	0.0	Normalised protein content score (0–1)
-	fat_score	float	0.0	Normalised fat content score (0–1)
-	calorie_score	float	0.0	Normalised calorie density score (0–1)
-	sugar_score	float	0.0	Normalised sugar content score (0–1)
-	spice_level_score	float	0.0	Spiciness score (0–1)
-	preparation_time_score	float	0.0	Estimated preparation time score (0–1)
-	preparation_difficulty_score	float	0.0	Estimated difficulty score (0–1)
-	meal_class	str None	None	High-level meal classification label
-	preparation_and_cooking_suggestion	tuple	(None,)	AI-generated cooking tip; populated post-build
-	serving_suggestion	tuple	(None,)	AI-generated serving tip; populated post-build
-	safe_keeping_suggestion	tuple	(None,)	AI-generated storage tip; populated post-build
-	steps	list	[]	Ordered list of AI-generated step-by-step cooking instructions

Module-level functions:

Signature	Return	Description	Exceptions
build(query_result: dict) -> Recipe	Recipe	Factory function. Maps all fields from a database query result dict onto a new Recipe instance and calls get_steps() to populate recipe.steps via the AI model.	KeyError if required field missing from query_result
get_steps(recipe: Recipe) -> list	list	Instantiates AIUtils() with the default model and calls model.get_steps(name, ingredients,	Exceptions from AIUtils propagate to caller

		preparation) to generate ordered cooking steps.	
--	--	---	--

3.4.3 Batch Ingestion Pipeline — row_adder.py

A standalone script that reads raw scraped recipe data from a JSONL file, enriches each recipe with AI-generated flags and embeddings using AIUtils, validates the required fields, and bulk-inserts the results into the SQLite database. It does not define a class; its logic is split across two module-level functions and a `__main__` block.

Design pattern applied: Pipeline / Batch Processor — data flows linearly through discrete stages (read → flag → validate → embed → persist), with each stage implemented as a separate function.

Module-level functions:

Signature	Return	Description	Exceptions
<code>serialize_f32(vector: list) -> bytes</code>	bytes	Serialises a Python list of floats into a compact little-endian binary format using <code>struct.pack</code> . Used to prepare embedding vectors for insertion into the <code>recipes_vec</code> virtual table.	<code>struct.error</code> if vector contains non-float values
<code>_save_to_database(recipes: list[dict])</code>	None	Opens a SQLite connection, loads <code>sqlite_vec</code> , and inserts each recipe dict into the <code>recipes</code> table (metadata fields) and <code>recipes_vec</code> table (binary embedding). Commits all inserts atomically.	<code>sqlite3.Error</code> on insert failure; connection always closed via explicit <code>conn.close()</code>
<code>_recipe_categorizer(recipes: list[dict])</code>	None	Stub function reserved for future categorisation logic. Currently a no-op pass.	None (stub)

Ingestion Pipeline — `__main__` execution flow:

Step	Action	Detail
1	Initialise AIUtils	Instantiates AIUtils with <code>gemini-2.5-flash-lite</code> model; records start time for performance logging
2	Count JSONL lines	Opens <code>scraped_recipes.jsonl</code> , counts total lines, resets file pointer to beginning
3	Batch loop (5 recipes/batch)	Iterates over JSONL in batches of 5; deserialises each line with <code>json.loads()</code>
4	Add flags	Calls <code>flag_adder._add_flags(recipes)</code> to obtain AI-generated metadata flags (dietary, nutritional scores, <code>meal_type</code> , etc.) for the batch

5	Write progress	Writes current count to progress.txt after each batch for resumability
6	Merge and validate	Merges recipe dict with flags dict; checks all 25 required fields are present and non-None
7	Persist to DB	Calls <code>_save_to_database(recipes_with_flags)</code> to insert validated recipes
8	Set embeddings	Calls <code>flag_adder.set_embeddings()</code> to generate and store vector embeddings for the batch
9	Log and clean up	Prints elapsed time; sets recipes and flags to None to free memory

Required fields validated before database insertion (all 25 must be present and non-None):

Category	Fields
Identity	name, ingredients, preparation, embedding
Classification	meal_type, main_ingredient, nationality, meal_class
Dietary flags	vegetarian, vegan, gluten_free, dairy_free, nut_free, high_fiber
Goal flags	weight_loss_diet, muscle_gain_bulking, balanced_healthy
Nutritional scores	carb_score, protein_score, fat_score, calorie_score, sugar_score, spice_level_score
Preparation scores	preparation_time_score, preparation_difficulty_score

3.4.4 Database Schema

The SQLite database (recipes.db) contains two tables managed by this subsystem:

Table	Type	Primary Key	Description
recipes	Standard table	id (INTEGER)	Stores all recipe metadata: identity, dietary flags, nutritional scores, and preparation details
recipes_vec	Virtual table (sqlite_vec)	rowid = recipes.id	Stores binary-serialised float32 embedding vectors; enables cosine/L2 similarity search across recipes

3.4.5 Class and Module Relationships

Relationship	From	To	Description
--------------	------	----	-------------

Uses	row_adder (__main__)	Database (implicit)	row_adder bypasses the Database class and writes directly via sqlite3; Database is used for reads
Uses	recipe_manager.build()	Database.get_recipe()	build() receives a query result dict from Database.get_recipe() as its input
Creates	recipe_manager.build()	Recipe	Factory function instantiates and fully populates a Recipe object
Dependency	recipe_manager.get_steps()	ai_logic.AIUtils	Instantiates AIUtils to call get_steps(name, ingredients, preparation)
Dependency	row_adder.__main__	ai_logic.AIUtils	Instantiates AIUtils(gemini-2.5-flash-lite) for _add_flags() and set_embeddings()
Data flow	row_adder._save_to_database()	recipes table	Inserts metadata fields for each validated recipe
Data flow	row_adder._save_to_database()	recipes_vec table	Inserts binary-serialised embedding vector keyed by recipe rowid

Design Patterns Summary:

Pattern	Where Applied	Justification
Repository	Database class	Encapsulates all raw SQL and connection management; callers use get_recipe() and execute_query() without knowing connection lifecycle or extension loading details
Factory Method	recipe_manager.build()	Separates Recipe object construction from its representation; the Recipe class stays a pure data object while build() handles mapping, type conversion, and AI enrichment
Pipeline / Batch Processor	row_adder.__main__	Data flows through discrete, ordered stages (read → flag → validate → persist → embed) each implemented as a separate function, making each stage independently testable

External Dependencies:

Library / Module	Source	Role
sqlite3	Python stdlib	Core relational database engine for recipe storage and retrieval
sqlite_vec	PyPI (third-party)	SQLite extension enabling vector similarity search on the recipes_vec virtual table
struct	Python stdlib	Binary serialisation of float32 embedding vectors via struct.pack
ai_logic.AIUtils	Internal module	Generates dietary flags, nutritional scores, embeddings, and cooking steps via LLM calls
read_database	Internal module	Provides serialize_bytes() for deserialising binary embedding fields on read
json	Python stdlib	Deserialises JSONL recipe records during batch ingestion

[Insert Sequence Diagram for key use cases of Component 1 Name here]

4. Glossary

All technical and domain-specific terms used throughout this document are defined here in alphabetical order.

Term	Definition
Agent	An AI component that receives input, decides which action to take, and invokes one or more tools or sub-agents to produce a result. In Chef Bono, a Gemini-backed agent handles intent detection, recipe retrieval, and conversational guidance.
asyncio	Python's standard-library framework for writing concurrent code using async/await syntax. Used in the TTS module to stream audio synthesis requests to Microsoft Edge TTS without blocking the main thread.
Batch Ingestion	The offline process of reading raw recipe records from a JSONL file in groups (batches of 5), enriching each batch with AI-generated flags and embeddings, validating required fields, and persisting results to SQLite.
Build Type	A string token passed to AIUtils._get_build() that selects a specific system instruction and JSON response schema for a Gemini API call (e.g., "talk", "query", "build_recipe").
Context Window	The maximum number of tokens (input + output combined) that a language model can process in a single call. Managed per Chat instance in Chef Bono to prevent exceeding the model's capacity.

Edge TTS	Microsoft's cloud-based neural text-to-speech service, accessed via the edge-tts Python library. Produces MP3 audio from text using a specified neural voice (tr-TR-AhmetNeural in this project).
Embedding	A dense, fixed-length numerical vector that encodes the semantic meaning of a piece of text. Chef Bono generates 256-dimensional embeddings (PCA-reduced from 3072) for each recipe to enable vector similarity search.
Facade	A structural design pattern that provides a simplified interface to a complex subsystem. Used in AIUtils, BasicInterface, and SpeechToTextService to hide multi-step internal logic behind a small number of public methods.
Factory Method	A creational design pattern in which a dedicated function (build()) constructs and returns an object, keeping the class free of construction logic. Used in recipe_manager.py to build Recipe instances from database query results.
Function Calling	A Gemini API feature that lets the model output a structured call to a predefined tool (e.g., generate_sql, select_recipe) instead of a plain text response, enabling deterministic action routing.
Gemini	Google's family of large language models accessed via the google-genai SDK. Chef Bono uses gemini-2.5-flash-lite as the default model for text generation, function calling, and image analysis.
JSONL	JSON Lines — a file format in which each line is a self-contained JSON object. Used to store raw scraped recipe records in scraped_recipes.jsonl before ingestion into SQLite.
LLM	Large Language Model — a neural network trained on large text corpora capable of generating and reasoning about natural language. Chef Bono uses Google Gemini LLMs for all AI-powered features.
PCA	Principal Component Analysis — a technique that reduces high-dimensional vectors to fewer dimensions while preserving variance. Used to compress 3072-dimensional Gemini embeddings to 256 dimensions, saving database storage.
Recipe	The core domain object in Chef Bono, representing a single dish with its identity, ingredients, preparation text, dietary flags, nutritional scores, and AI-generated step-by-step cooking instructions.
Repository	A design pattern that encapsulates data-access logic behind a stable interface, decoupling business logic from storage details. Implemented by the Database class (db_query_handler.py).
SQLite	A serverless, file-based relational database engine built into Python's standard library (sqlite3). Chef Bono stores all recipe metadata in a single file, recipes.db.
sqlite_vec	A third-party SQLite extension that adds a virtual table (recipes_vec) supporting cosine and L2 vector similarity search over stored float32 embedding vectors.
STT	Speech-to-Text — converting spoken audio into a text string. Implemented in stt.py using the OpenAI Whisper base model with the Turkish (tr) language setting.

TTS	Text-to-Speech — synthesising spoken audio from a text string. Implemented in tts.py using Microsoft Edge TTS with the tr-TR-AhmetNeural neural voice.
Token	The basic unit of text processed by a language model, roughly a word or word-fragment. Token counts determine API cost and context-window usage; AIUtils tracks input and output token limits per Gemini model.
Vector Search	A retrieval technique that finds records whose stored embeddings are closest (cosine or L2) to a query embedding. Used to match user requests to semantically relevant recipes in the recipes_vec table.
Whisper	An open-source automatic speech recognition model by OpenAI (openai-whisper on PyPI). Used in the STT module to transcribe microphone recordings to text locally, without a network call.

5. References

The following sources were consulted during the object design phase:

1. Bruegge, B. and Dutoit, A.H. (2004). Object-Oriented Software Engineering, Using UML, Patterns, and Java (2nd ed.). Prentice-Hall. ISBN: 0-13-047110-0.
2. ACM Code of Ethics and Professional Conduct. Association for Computing Machinery. <https://www.acm.org/code-of-ethics>
3. IEEE Std 1016-2009 — IEEE Standard for Information Technology: Systems Design — Software Design Descriptions.
4. HLD document— Chef Bono Team, CMPE 491, 2025
5. Anthropic. (2025). Claude (Claude 4.6 Sonnet version) [Large language model]. <https://claude.ai>
6. Python Software Foundation – <https://docs.python.org>
7. SQLite Documentation – <https://www.sqlite.org/docs.html>
8. Google Gemini API – <https://ai.google.dev/gemini-api/docs>
9. Sqlite-vec - <https://alexgarcia.xyz/sqlite-vec/installation.html>
10. Draw.io - <https://app.diagrams.net/>